

STUDENT CODE: **306D**

Gebze Technical University
Department of Computer Engineering
CSE 241/501
Fall 2024
Midterm Exam Nov 13th 2024

Student Code	Q1 (25)	Q2 (30)	Q3 (45)	Exam Format (-10)
306				

Read the instructions below carefully

- Do not write your name or your student number in any part of the exam materials including the answer sheets.
- Your individual student codes are already printed on your question and answer sheets. Keep this student code until the exam ends. You will use it to sign the exam attendance sheet.
- Answer each question on the designated page in the box only. We will not (and cannot) grade any materials outside of the designated boxes
- Follow the line guides when writing, do not skip lines, do not use more than one line for a single line.
- Hand over your cell phones, books, and notes to the TA before the exam starts. You will not use any electronic devices or books during the exam.
- This exam has 3 questions; it will be graded out of 100 points. You have 120 minutes. If you do not follow any of the rules above, you will lose 10 points.

Question 1

- What is wrong with the following line? Explain our answer.
`assert( a++ >= 0)`
- What is wrong with the following copy constructor declaration of the class **MyClass**? Explain your answer.
`MyClass(const MyClass o);`
- What are the similarities and differences between global variables and static data members of classes?

Question 2

Implement a global C++ function that takes a 2-dimensional **std::vector** of **Rational** objects as parameter. Your function returns a 1-dimensional **vector** of **Rational** objects, where each element of this **vector** is the median (Turkish ortanca) of the elements of a row. Relevant parts of class **Rational** is given below. You may write additional helper functions. Do not implement any classes. Do not use any library functions for sorting.

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1); // Assume a constructor exists
    bool operator<(const Rational& rhs) const; // Compares two Rationals
    Rational operator+(const Rational& rhs) const;
    Rational operator/(int value) const;

    // Other necessary member functions and data here
};
```

In case you need, the median is the middle value in a list of numbers sorted in ascending order, or the average of the two middle numbers if the list contains an even number of elements.

STUDENT CODE: **306D**

Question 3

Study the following driver code for a general-purpose reusable class named **GTUStack** whose output is given on the right.

```
#include <iostream>
#include "GTUIntStack.h"
using std::cout;

int main() {
    GTUIntStack stack;
    stack.push(1);
    stack.push(2);
    stack.push(3);
    cout << stack << std::endl;

    GTUIntStack stackCopy(stack);
    stackCopy.push(4);
    cout << stackCopy << std::endl;

    cout << "Size of original stack: "
         << stack.size() << std::endl;
    cout << "Size of copied stack: "
         << stackCopy.size() << std::endl;

    if (!stack.isEmpty())
        cout << "Original stack is not
empty."
             << std::endl;

    stack.pop();
    stack.pop();
    stack.pop();

    if (stack.isEmpty())
        cout << "Original stack is now
empty."
             << std::endl;

    cout << GTUIntStack::getStacksCreated()
         << std::endl;

    return 0;
}
```

Output Screen

```
3 2 1
4 3 2 1
Size of original stack: 3
Size of copied stack: 4
Original stack is not empty.
Original stack is now empty.
2
```

Class **GTUStack** has three data members only: an **int *** to keep the integers, an **int** to keep the capacity of the stack, and another **int** to show the top of the stack.

Q3 Part a) Write the complete class header. You should not implement any functions in this part.

Q3 Part b) Write the implementation file that includes all the necessary class member and global functions. Do not forget to check for errors.

STUDENT CODE: **306D**

Write your answer for **Question 1** in the box below.

a)

```
assert(a++ > 0)
```

For this code line problem is ++ operator. Because before it takes a and after look the condition so if we give the 0 for a it will give problem and exit the program. But if write like `assert(++a > 0)` this code will take a before increment a and look the condition. So a++ is the wrong in this code line. So this is not effective code. We must change it like I say above.

b)

```
MyClass(const MyClass o)
```

In this code line we don't take reference of MyClass o member. This is copy of that o so if we do any changing on MyClass member it will do just in the MyClass copy constructor.

If we write like `MyClass(const MyClass& o)` we will take reference of o and we can reach the where is the o's memory and we can work more efficiently.

finally if we want to declare copy constructor we must take reference of other object to work more efficiently.

c)

Similarities of global variables and static data members are both are reachable. For static data members all class functions can reach that member. And the likely global variables can reach that by all functions not just for class functions.

And the differences just class members and functions can reach static member directly for global variables all of functions can reach that.

And when declare this two element we use static adjective for static member.

When declare global variables just use the type of element

STUDENT CODE: **306D**

Write your answer for **Question 2** in the box below. Use two column format, do not write anywhere outside the box. Follow the line guides when writing.

```
void CreateMedian(const vector<vector<Rational>> x) {  
    if (x.size() == 0 || x[0].size() == 0) exit(-1);  
  
    vector<Rational> v;  
  
    for (int i = 0; i < x.size(); i++) {  
        for (int j = 0; j < x[0].size(); j++) {  
            if (!x[i][j] < x[i+1][j]) {  
                Rational temp = x[i][j];  
                x[i][j] = x[i+1][j];  
                x[i+1][j] = temp;  
            }  
        }  
    }  
  
    for (int l = 0; l < x.size(); l++) {  
        if (x.size() % 2 == 0) {  
            int m = x.size() / 2;  
            v[l] = (x[m] + x[m+1]) / 2;  
            break;  
        }  
        if (x.size() % 2 != 0) {  
            int a = x.size() / 2;  
            v[l] = x[a+1];  
            break;  
        }  
    }  
}
```

STUDENT CODE: **306D**

Write your answer for **Question 3-a** in the box below.

```
#ifndef stack_h
#define stack_h
using namespace std;
#include <cstdlib>
class GTUIntStack {
private:
    int* data;
    int capacity;
    int numUsed;
static int StackCreated;
public:
    Stack(int num=0, cap=10),
    GTUIntStack(const GTUIntStack& o); // copy constructor
    ~GTUIntStack(int num=0, cap=10); // copy constructor
    GTUIntStack(const GTUIntStack& o); // copy constructor
    ~GTUIntStack(); // destructor
    void push(int x);
    void pop();
    bool isEmpty();
    int getNumUsed() const;
    int getCapacity() const;
    int size() const;

    friend ostream& operator<< (ostream& out, const GTUIntStack& o);
    int getStacksCreated();
};
#endif
```

STUDENT CODE: **306D**

Write your answer for **Question 3-b** in the box below.

```
#include "GTUIntStack.h"
int GTUIntStack::StackCreated = 0;
GTUIntStack::GTUIntStack(int num=0, int cop=10)
: numUsed(num), copacity(cop), data(nullptr) {
    ++StackCreated;
}
GTUIntStack::GTUIntStack(const GTUIntStack& o): copacity
: copacity(o.copacity), numUsed(o.numUsed)
{
    data = new int[copacity];
    for (int i=0; i<numUsed; i++) {
        data[i] = o.data[i];
    }
    ++StackCreated;
}
GTUIntStack::~~GTUIntStack() {
    delete [] data;
    --StackCreated;
}
void GTUIntStack::push(int x) {
    if (numUsed == copacity) {
        copacity *= 2;
        int* temp = new int[copacity];
        for (int i=0; numUsed; i++) {
            temp[i] = data[i];
        }
        delete [] data;
        data = temp;
    }
    data[numUsed++] = x;
}
void GTUIntStack::pop() {
    if (numUsed == 0) exit(-1);
    --numUsed;
}
bool GTUIntStack::isEmpty() {
    if (numUsed == 0) return true;
    else {
        return false;
    }
}
```

STUDENT CODE: **306D**

Continue your answer for **Question 3-b** in the box below.

```
int GTUIntStack::getNumUsed() const {  
    return numUsed;  
}  
  
int GTUIntStack::getCapacity() const {  
    return capacity;  
}  
  
int GTUIntStack::size() {  
    return 'getNumUsed();  
}  
  
ostream & operator<<(ostream & out, const GTUIntStack & o) {  
    for(int i=0; i< o.size(); i++) {  
        out << data[i] << " ";  
    }  
    return out;  
}  
  
int GTUIntStack::getStacksCreated() {  
    return stackCreated;  
}
```

STUDENT CODE: **306D**

Continue your answer for **Question 3-b** in the box below.

Blank lined paper for writing answers.